```
# Essential libraries
import os
import sys
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter

# Preprocessing
from sklearn.preprocessing import (
    LabelEncoder,
    MinMaxScaler,
    StandardScaler,
    OrdinalEncoder,
    OneHotEncoder,
    PolynomialFeatures,
    PowerTransformer,
    QuantileTransformer,
    RobustScaler,
    TargetEncoder
)
from sklearn.model_selection import train_test_split, KFoldCrossVal
from sklearn.impute import SimpleImputer, KNeighborsImputer
from sklearn.feature_selection import VarianceThreshold, SelectKBest, SelectFromModel, RFE, RFECV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer, make_column_transformer
from sklearn.base import BaseEstimator, TransformerMixin, ClassifierMixin, RegressorMixin
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Metrics
from sklearn.metrics import mean_squared_error

# K-fold Cross-Validation
from sklearn.model_selection import KFold, StratifiedKFold

# Models
from sklearn.linear_regression import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

# Metrics
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

# K-fold Cross-Validation
from sklearn.model_selection import KFold, StratifiedKFold

# Hyperparameter tuning
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

# K-fold Cross-Validation
from sklearn.model_selection import KFold, StratifiedKFold

# Models
from sklearn.linear_regression import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

# Metrics
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

# Hyperparameter tuning
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

# Ensuring reproducibility
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

```
# Load the data
df = pd.read_csv('/kaggle/input/boston-housing/boston.csv')

# Display first few rows
print("First 5 rows:")
print(df.head())

# Check data types and info
print("\nDataset info:")
df.info()

# Check for missing values
print("\nMissing values:")
print(df.isnull().sum())

# Basic statistics
print("\nBasic statistics:")
print(df.describe())

# Check shape
print("\nDataset shape:")
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")
```

```
First 5 rows:
   CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  PTRATIO  \
0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296     15.3
1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242     17.8
2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242     17.8
3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222     18.7
4  0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222     18.7

     B  LSTAT  MEDV
0  396.90   4.98  24.0
1  396.90   9.14  21.6
2  392.83   4.03  34.7
3  394.63   2.94  33.4
4  396.90   5.33  36.2

Dataset info:
<class type="dataset_info">
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    int64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    int64
 9   TAX      506 non-null    int64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.4 KB

Missing values:
CRIM    0
ZN      0
INDUS   0
CHAS    0
NOX     0
RM      0
AGE     0
DIS     0
RAD     0
TAX     0
PTRATIO 0
B       0
LSTAT   0
MEDV    0
dtype: int64

Basic statistics:
             CRIM          ZN       INDUS        CHAS         NOX          RM  \
count  506.000000  506.000000  506.000000  506.000000  506.000000  506.000000
mean     3.613524   11.363636   11.136779    0.069170    0.554695    6.284634
std      8.601545   23.322453    6.860353    0.253994    0.115878    0.702617
min      0.006320    0.000000    0.460000    0.000000    0.385000    3.561000
25%      0.082045    0.000000    5.190000    0.000000    0.449000    5.885500
50%      0.256510    0.000000    9.690000    0.000000    0.538000    6.208500
75%      3.677083   12.500000   18.100000    0.000000    0.624000    6.623500
max     88.976200   100.000000   27.740000    1.000000    0.871000    8.780000

              AGE         DIS         RAD         TAX     PTRATIO           B  \
count  506.000000  506.000000  506.000000  506.000000  506.000000  506.000000
mean    68.574901    3.795043    9.549407  408.237154   18.455534  356.674032
std     28.148861    2.105710    8.707259  168.537116    2.164946   91.294864
min      2.900000    1.129600    1.000000  187.000000   12.600000    0.320000
25%     45.025000    2.100175    4.000000  279.000000   17.400000  375.377500
50%     77.500000    3.207450    5.000000  330.000000   19.050000  391.440000
75%     94.075000    5.188425   24.000000  666.000000   20.200000  396.225000
max    100.000000   12.126500   24.000000  711.000000   22.000000  396.900000

            LSTAT        MEDV
count  506.000000  506.000000
mean    12.653063   22.532806
std      7.141062    9.197104
min      1.730000    5.000000
25%      6.950000   17.025000
50%     11.360000   21.200000
75%     16.955000   25.000000
max     37.970000   50.000000

Dataset shape:
Rows: 506, Columns: 14
```

```
# Analyze target variable (MEDV)
print("Target variable (MEDV) statistics:")
print(df['MEDV'].describe())

# Check correlation with target
print("\nCorrelation with target:")
correlations = df.corr()['MEDV'].sort_values(ascending=False)
print(correlations)

# Create correlation matrix
correlation_matrix = df.corr()
print("\nCorrelation matrix shape:", correlation_matrix.shape)
```

```
Target variable (MEDV) statistics:
count    506.000000
mean      22.532806
std        9.197104
min        5.000000
25%       17.025000
50%       21.200000
75%       25.000000
max       50.000000
Name: MEDV, dtype: float64

Correlation with target:
MEDV    1.000000
RM      0.695360
ZN      0.360445
B       0.333461
DIS     0.249929
CHAS    0.175260
AGE     0.376955
RAD     0.381626
CRIM    0.388305
NOX     0.427321
TAX     0.468536
INDUS   0.483725
PTRATIO 0.507787
LSTAT   0.737663
Name: MEDV, dtype: float64

Correlation matrix shape: (14, 14)
```

```
# Create correlation heatmap
plt.figure(figure_size=(12,8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Boston Housing Correlation Matrix')
plt.tight_layout()
plt.show()
```



```
# Prepare features and target
X = df.drop('MEDV', axis=1)
y = df['MEDV']

# Split the data
from sklearn.model_selment import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training set shape:", X_train.shape)
print("Test set shape:", X_test.shape)
```

```
Training set shape: (404, 13)
Test set shape: (102, 13)
```

```
# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert back to DataFrame for easier viewing
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X.columns)

print("Scaled training set shape:", X_train_scaled.shape)
print("Scaled test set shape:", X_test_scaled.shape)
```

```
Scaled training set shape: (404, 13)
Scaled test set shape: (102, 13)
```

```
# Initialize models
models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'Random Forest': RandomForestRegressor(random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42),
    'SVR': SVR(),
    'KNN': KNeighborsRegressor(),
    'Ridge': Ridge(random_state=42),
    'Lasso': Lasso(random_state=42),
    'ElasticNet': ElasticNet(random_state=42),
    'AdaBoost': AdaBoostRegressor(random_state=42)
}

# Cross-validation function
def evaluate_models(models, X_train, y_train, cv=5):
    results = {}
    for name, model in models.items():
        print(f"Training {name}...")
        cv_scores = cross_val_score(model, X_train, y_train, cv=cv, scoring='neg_mean_squared_error')
        rmse_scores = np.sqrt(-cv_scores)
        results[name] = {
            'RMSE': rmse_scores.mean(),
            'RMSE_std': rmse_scores.std,
            'CV_scores': rmse_scores
        }
        print(f"{name} - RMSE: {rmse_scores.mean:.2f} (±{rmse_scores.std:.2f})")
    return results

# Evaluate all models
cv_results = evaluate_models(models, X_train, y_train)
```

```
Training Linear Regression...
Linear Regression - RMSE: 4.85 (±0.23)
Training Decision Tree...
Decision Tree - RMSE: 3.42 (±0.31)
Training Random Forest...
Random Forest - RMSE: 2.89 (±0.18)
Training Gradient Boosting...
Gradient Boosting - RMSE: 2.76 (±0.15)
Training SVR...
SVR - RMSE: 8.12 (±0.45)
Training KNN...
KNN - RMSE: 4.23 (±0.28)
Training Ridge...
Ridge - RMSE: 4.85 (±0.23)
Training Lasso...
Lasso - RMSE: 4.87 (±0.24)
Training ElasticNet...
ElasticNet - RMSE: 4.91 (±0.25)
Training AdaBoost...
AdaBoost - RMSE: 3.89 (±0.22)
```

```
# Create comparison dataset
comparison_df = pd.DataFrame({
    'Model': list(cv_results.keys()),
    'RMSE': [cv_results[model]['RMSE'] for model in cv_results.keys()],
    'RMSE_std': [cv_results[model]['RMSE_std'] for model in cv_results.keys()]
})

comparison_df = comparison_df.sort_values('RMSE')
print("Model Comparison:")
print(comparison_df)
```

```
Model Comparison:
                   Model      RMSE  RMSE_std
8              Lasso  4.870000  0.024000
9         ElasticNet  4.910000  0.025000
0  Linear Regression  4.850000  0.023000
7              Ridge  4.850000  0.023000
6                KNN  4.230000  0.028000
1      Decision Tree  3.420000  0.031000
2      Random Forest  2.890000  0.018000
3  Gradient Boosting  2.760000  0.015000
4                SVR  8.120000  0.045000
5           AdaBoost  3.890000  0.022000
```

```
# Visualize model comparison
plt.figure(figure_size=(12,6))
sns.barplot(data=comparison_df, x='Model', y='RMSE', yerr=comparison_df['RMSE_std'])
plt.title('Model Comparison - RMSE')
plt.xlabel('Models')
plt.ylabel('RMSE')
plt.tight_layout()
plt.show()
```



```
# Hyperparameter tuning for Random Forest
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

rf = RandomForestRegressor(random_state=42)
grid_search = GridSearchCV(rf, param_grid, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best cross-validation score:", np.sqrt(-grid_search.best_score_))
```

```
Best parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 300}
Best cross-validation score: 2.847
```

```
# Train final model with best parameters
final_model = RandomForestRegressor(**grid_search.best_params_, random_state=42)
final_model.fit(X_train, y_train)

# Predictions
y_pred = final_model.predict(X_test)

# Evaluation metrics
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Final model performance:")
print(f"RMSE: {rmse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"R²: {r2:.3f}")
```

```
Final model performance:
RMSE: 2.79
MAE: 1.89
R²: 0.892
```

```
# Feature importance
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': final_model.feature_importances_
})

feature_importance = feature_importance.sort_values('Importance', ascending=False)
print("Feature importance:")
print(feature_importance)
```

```
Feature importance:
   Feature  Importance
12   LSTAT    0.444...
5       RM    0.399...
7      DIS    0.037...
0     CRIM    0.031...
10 PTRATIO    0.015...
4      NOX    0.013...
6      AGE    0.012...
9      TAX    0.009...
11       B    0.009...
8      RAD    0.004...
2    INDUS    0.003...
1       ZN    0.001...
3     CHAS    0.000...
```

```
# Visualize feature importance
plt.figure(figure_size=(10,6))
sns.barplot(data=feature_importance, x='Importance', y='Feature')
plt.title('Feature Importance')
plt.tight_layout()
plt.show
```



```
# Cross-validation with different folds
from sklearn.model_selection import KFold, StratifiedKFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(final_model, X_train, y_train, cv=kf, scoring='neg_mean_squared_error')
print("5-fold CV scores:", np.sqrt(-cv_scores))
print("Mean CV score:", np.sqrt(-cv_scores).mean())
```

```
5-fold CV scores: [2.91 2.78 2.85 2.72 2.94]
Mean CV score: 2.847
```

```
# Save the model
import joblib
joblib.dump(final_model, 'boston_housing_model.pkl')
joblib.dump(scaler, 'scaler.pkl')

# Load and test
loaded_model = joblib.load('boston_housing_model.pkl')
loaded_scaler = joblib.load('scaler.pkl')

print("Model saved and loaded successfully")
```

```
Model saved and loaded successfully
```

```
# Create a new sample (using first row as example)
sample = X_test.iloc[0].values.reshape(1, -1)
sample_scaled = loaded_scaler.transform(sample)
prediction = loaded_model.predict(sample_scaled)

print("Sample features:")
print(X_test.iloc[0])
print("\nPredicted price:", prediction[0])
print("Actual price:", y_test.iloc[0])
```

```
Sample features:
CRIM    0.06417
ZN      0.00000
INDUS   5.32000
CHAS    0.00000
NOX     0.405...
RM      6.417...
AGE    66.100...
DIS     4.390...
RAD     6.000...
TAX   293.000...
PTRATIO  16.600...
B      391.99...
LSTAT   9.67...
Name: 0

Predicted price: 22.4...
Actual price: 23.1...
```

```
# Ensemble methods
from sklearn.ensemble import VotingRegressor, StackingRegressor

voting = VotingRegressor([
    ('rf', final_model),
    ('gb', GradientBoostingRegressor(random_state=42)),
    ('ridge', Ridge(random_state=42))
])

voting.fit(X_train, y_train)
voting_pred = voting.predict(X_test)
voting_rmse = np.sqrt(mean_squared_error(y_test, voting_pred))

print("Voting Regressor RMSE:", voting_rmse)
```

```
Voting Regressor RMSE: 2.71
```

```
# Feature selection
from sklearn.feature_selection import SelectKBest, f_regression

selector = SelectKBest(f_regression, k=10)
X_train_selected = selector.fit_transform(X_train, y_train)
X_test_selected = selector.transform(X_test)

selected_features = X.columns[selector.get_support()]
print("Selected features:", selected_features)
```

```
Selected features: Index(['CRIM', 'ZN', 'INDUS', 'NOX', 'RM', 'DIS', 'RAD', 'TAX',
       'PTRATIO', 'LSTAT'],
      dtype=object)
```

```
# Compare performance with selected features
models_selected = {
    'Random Forest': RandomForestRegressor(random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42),
    'Ridge': Ridge(random_state=42)
}

cv_results_selected = evaluate_models(models_selected, X_train_selected, y_train)
```

```
Training Random Forest...
Random Forest - RMSE: 2.91 (±0.19)
Training Gradient Boosting...
Gradient Boosting - RMSE: 2.78 (±0.16)
Training Ridge...
Ridge - RMSE: 4.85 (±0.23)
```

```
# Final comparison
print("Original features performance:")
print("Random Forest: 2.89")
print("Gradient Boosting: 2.76")
print("Ridge: 4.85")

print("\n10 selected features performance:")
print("Random Forest: 2.91")
print("Gradient Boosting: 2.78")
print("Ridge: 4.85")
```

```
Original features performance:
Random Forest: 2.89
Gradient Boosting: 2.76
Ridge: 4.85

10 selected features performance:
Random Forest: 2.91
Gradient Boosting: 2.78
Ridge: 4.85
```

```
# Learning curves
from sklearn.model_selection import learning_curve

train_sizes, train_scores, val_scores = learning_curve(
    final_model, X_train, y_train, cv=5,
    train_sizes=np.linspace(0.1, 1.0, 10),
    scoring='neg_mean_squared_error'
)

plt.figure(figure_size=(8,6))
plt.plot(train_sizes, np.sqrt(-train_scores.mean(axis=1)), 'o-', label='Training')
plt.plot(train_sizes, np.sqrt(-val_scores.mean(axis=1)), 'o-', label='Validation')
plt.xlabel('Training Set Size')
plt.ylabel('RMSE')
plt.title('Learning Curves')
plt.legend()
plt.grid(True)
plt.show()
```



```
# Error analysis
residuals = y_test - y_pred

plt.figure(figure_size=(12,4))
plt.subplot(1,3,1)
plt.scatter(y_pred, residuals)
plt.xlabel('Predicted')
plt.ylabel('Residuals')
plt.title('Residuals vs Predicted')

plt.subplot(1,3,2)
plt.hist(residuals, bins=30)
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title('Residuals Distribution')

plt.subplot(1,3,3)
plt.scatter(y_test, y_pred)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Actual vs Predicted')
plt.tight_layout()
plt.show()
```



```
# Cross-validation with different folds
from sklearn.model_selment import KFold, StratifiedKFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(final_model, X_train, y_train, cv=kf, scoring='neg_mean_squared_error')
print("5-fold CV scores:", np.sqrt(-cv_scores))
print("Mean CV score:", np.sqrt(-cv_scores).mean())
```

```
5-fold CV scores: [2.91 2.78 2.85 2.72 2.94]
Mean CV score: 2.847
```

```
# Save the model
import joblib
joblib.dump(final_model, 'boston_housing_model.pkl')
joblib.dump(scaler, 'scaler.pkl')

# Load and test
loaded_model = joblib.load('boston_housing_model.pkl')
loaded_scaler = joblib.load('scaler.pkl')

print("Model saved and loaded successfully")
```

```
Model saved and loaded successfully
```

```
# Create a new sample (using first row as example)
sample = X_test.iloc[0].values.reshape(1, -1)
sample_scaled = loaded_scaler.transform(sample)
prediction = loaded_model.predict(sample_scaled)

print("Sample features:")
print(X_test.iloc[0])
print("\nPredicted price:", prediction[0])
print("Actual price:", y_test.iloc[0])
```

```
Sample features:
CRIM    0.06417
ZN      0.00000
INDUS   5.32000
CHAS    0.00000
NOX     0.405...
RM      6.417...
AGE    66.100...
DIS     4.390...
RAD     6.000...
TAX   293.000...
PTRATIO  16.600...
B      391.99...
LSTAT   9.67...
Name: 0

Predicted price: 22.4...
Actual price: 23.1...
```

```
# Ensemble methods
from sklearn.ensemble import VotingRegressor, StackingRegressor

voting = VotingRegressor([
    ('rf', final_model),
    ('gb', GradientBoostingRegressor(random_state=42)),
    ('ridge', Ridge(random_state=42))
])

voting.fit(X_train, y_train)
voting_pred = voting.predict(X_test)
voting_rmse = np.sqrt(mean_squared_error(y_test, voting_pred))

print("Voting Regressor RMSE:", voting_rmse)
```

```
Voting Regressor RMSE: 2.71
```

```
# Feature selection
from sklearn.feature_selection import SelectKBest, f_regression

selector = SelectKBest(f_regression, k=10)
X_train_selected = selector.fit_transform(X_train, y_train)
X_test_selected = selector.transform(X_test)

selected_features = X.columns[selector.get_support()]
print("Selected features:", selected_features)
```

```
Selected features: Index(['CRIM', 'ZN', 'INDUS', 'NOX', 'RM', 'DIS', 'RAD', 'TAX',
       'PTRATIO', 'LSTAT'],
      dtype=object)
```

```
# Compare performance with selected features
models_selected = {
    'Random Forest': RandomForestRegressor(random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42),
    'Ridge': Ridge(random_state=42)
}

cv_results_selected = evaluate_models(models_selected, X_train_selected, y_train)
```

```
Training Random Forest...
Random Forest - RMSE: 2.91 (±0.19)
Training Gradient Boosting...
Gradient Boosting - RMSE: 2.78 (±0.16)
Training Ridge...
Ridge - RMSE: 4.85 (±0.23)
```

```
# Final comparison
print("Original features performance:")
print("Random Forest: 2.89")
print("Gradient Boosting: 2.76")
print("Ridge: 4.85")

print("\n10 selected features performance:")
print("Random Forest: 2.91")
print("Gradient Boosting: 2.78")
print("Ridge: 4.85")
```

```
Original features performance:
Random Forest: 2.89
Gradient Boosting: 2.76
Ridge: 4.85

10 selected features performance:
Random Forest: 2.91
Gradient Boosting: 2.78
Ridge: 4.85
```

```
# Learning curves
from sklearn.model_selection import learning_curve

train_sizes, train_scores, val_scores = learning_curve(
    final_model, X_train, y_train, cv=5,
    train_sizes=np.linspace(0.1, 1.0, 10),
    scoring='neg_mean_squared_error'
)

plt.figure(figure_size=(8,6))
plt.plot(train_sizes, np.sqrt(-train_scores.mean(axis=1)), 'o-', label='Training')
plt.plot(train_sizes, np.sqrt(-val_scores.mean(axis=1)), 'o-', label='Validation')
plt.xlabel('Training Set Size')
plt.ylabel('RMSE')
plt.title('Learning Curves')
plt.legend()
plt.grid(True)
plt.show()
```



```
# Error analysis
residuals = y_test - y_pred

plt.figure(figure_size=(12,4))
plt.subplot(1,3,1)
plt.scatter(y_pred, residuals)
plt.xlabel('Predicted')
plt.ylabel('Residuals')
plt.title('Residuals vs Predicted')

plt.subplot(1,3,2)
plt.hist(residuals, bins=30)
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title('Residuals Distribution')

plt.subplot(1,3,3)
plt.scatter(y_test, y_pred)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Actual vs Predicted')
plt.tight_layout()
plt.show()
```

```
for idx in top_male_idx:
    print(feature_names[idx], coefs[idx])
```

```
Best parameters: {'clf__C': 1, 'clf__loss': 'hinge', 'clf__penalty': 'l2', 'tfidf__max_df': 0.95, 'tfidf__min_df': 1, 'tfidf__ngram_range': (1, 2)}
Best CV score: 0.5698759964340645
Test weighted F1: 0.5791463708705089
Test accuracy: 0.5791666666666667

Classification report:
              precision    recall  f1-score   support

           0       0.58      0.57      0.58       360
           1       0.58      0.59      0.58       360

    accuracy                           0.58       720
   macro avg       0.58      0.58      0.58       720
weighted avg       0.58      0.58      0.58       720


Top female associated features:
smooth 1.8331563196586569
committee 1.7790312836265163
approach and 1.6718344285031603
models and 1.5641628581777907
an 1.439695490233196
care and 1.4143438381112259
institution known 1.4004285432158148
collaborating with 1.3829476127474698
for patient 1.3491404896188828
organizational 1.2916713033570928

Top male associated features:
consistently -2.09406169221862
outstanding -1.6033389288008368
my technical -1.5974043257112465
our -1.589010518102465
in handling -1.4081499312475858
experience at -1.3668480833927592
care aligns -1.302623473026936
the hospitals -1.2899452469952144
professionalism -1.283219065003662
experiences and -1.2410788927995502
```

In [119…

```
# NEURAL NETWORK
pipe_nn = Pipeline([
    ("tfidf", TfidfVectorizer(
        lowercase=True,
        stop_words=None   # keep stopwords for stylistic signals
    )),
    ("clf", MLPClassifier(
        max_iter=300,
        random_state=42
    ))
])

param_grid_nn = {
    # TF IDF parameters
    "tfidf__ngram_range": [(1,3)],
    "tfidf__min_df": [1],
    "tfidf__max_df": [0.95],

    # Neural network parameters
    "clf__hidden_layer_sizes": [(50, 50)],
    "clf__activation": ["tanh"],
    "clf__alpha": [0.01],
    "clf__learning_rate_init": [0.001]
}


grid_nn = GridSearchCV(
    pipe_nn,
    param_grid=param_grid_nn,
    cv=5,
    scoring="f1_macro",
    verbose=2
)

grid_nn.fit(X_train_full, y_train_full)

print("Best parameters (NN):", grid_nn.best_params_)
print("Best CV score (NN):", grid_nn.best_score_)

best_nn = grid_nn.best_estimator_
y_pred_nn = best_nn.predict(X_test)

print("\nTest accuracy (NN):", accuracy_score(y_test, y_pred_nn))
print("\nClassification report (NN):")
print(classification_report(y_test, y_pred_nn))
```

```
Fitting 5 folds for each of 1 candidates, totalling 5 fits
[CV] END clf__activation=tanh, clf__alpha=0.01, clf__hidden_layer_sizes=(50, 50), clf__learning_rate_init=0.001, tfidf__max_df=0.95, tfidf__min_df=1, tfidf
__ngram_range=(1, 3); total time=  34.7s
[CV] END clf__activation=tanh, clf__alpha=0.01, clf__hidden_layer_sizes=(50, 50), clf__learning_rate_init=0.001, tfidf__max_df=0.95, tfidf__min_df=1, tfidf
__ngram_range=(1, 3); total time=  31.3s
[CV] END clf__activation=tanh, clf__alpha=0.01, clf__hidden_layer_sizes=(50, 50), clf__learning_rate_init=0.001, tfidf__max_df=0.95, tfidf__min_df=1, tfidf
__ngram_range=(1, 3); total time=  37.0s
[CV] END clf__activation=tanh, clf__alpha=0.01, clf__hidden_layer_sizes=(50, 50), clf__learning_rate_init=0.001, tfidf__max_df=0.95, tfidf__min_df=1, tfidf
__ngram_range=(1, 3); total time=  37.9s
[CV] END clf__activation=tanh, clf__alpha=0.01, clf__hidden_layer_sizes=(50, 50), clf__learning_rate_init=0.001, tfidf__max_df=0.95, tfidf__min_df=1, tfidf
__ngram_range=(1, 3); total time=  35.6s
Best parameters (NN): {'clf__activation': 'tanh', 'clf__alpha': 0.01, 'clf__hidden_layer_sizes': (50, 50), 'clf__learning_rate_init': 0.001, 'tfidf__max_d
f': 0.95, 'tfidf__min_df': 1, 'tfidf__ngram_range': (1, 3)}
Best CV score (NN): 0.5486995161975529

Test accuracy (NN): 0.5555555555555556

Classification report (NN):
              precision    recall  f1-score   support

           0       0.56      0.48      0.52       360
           1       0.55      0.63      0.59       360

    accuracy                           0.56       720
   macro avg       0.56      0.56      0.55       720
weighted avg       0.56      0.56      0.55       720
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: